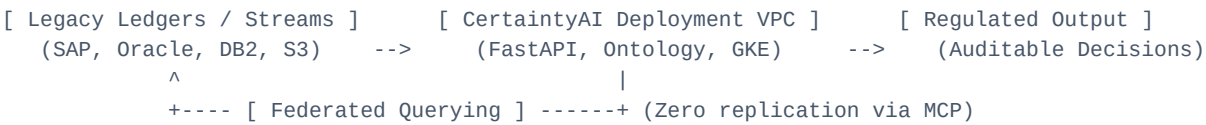


CertaintyAI - Enterprise Deployment Strategy & Production Readiness Guide

This guide establishes the comprehensive deployment strategy, production tech stack, and step-by-step instructions for hosting the **CertaintyAI** platform in highly regulated enterprise environments. It aligns directly with the **Google for Startups AI Agents Challenge - Track 3 (Business Ready)** parameters and satisfies the security, sovereignty, and data-governance requirements of mid-market healthcare, BFSI, and cybersecurity organizations.

1. Architectural Strategy: In-Place Governance & Zero-Replication

For regulated enterprises, copying or moving data is a compliance violation. CertaintyAI's deployment strategy centers on **in-place semantic orchestration**.



The core strategy leverages:

- Sovereign Single-Tenant VPCs:** Deploying CertaintyAI entirely within the client's own cloud perimeter (e.g., Google Cloud VPC) so that no data ever exits their trust boundary.
- Federated Query Engine (Layer 2 & 3):** Syncing directly to legacy transactional systems (SAP ERP, Oracle, DB2) and cloud data platforms (Snowflake) via secure Model Context Protocol (MCP) servers and read-only database connections.
- Cryptographic Lineage Logging (Layer 8):** Recording all agentic decisions, prompt contexts, and validation results directly into an immutable database to ensure full auditability.

2. Tech Stack for Enterprise Deployment

To move CertaintyAI from local prototype (FastAPI + SQLite + local React dev server) to enterprise production, the tech stack is upgraded as follows:

Layer	Component	Local/Dev
Experience	Web Application	Vite Dev Server
Orchestration	API Gateway	FastAPI direct
Logic	Core Backend	FastAPI / Uvicorn

Agentic	LLM Orchestration	Local Direct API
Data Plane	Relational Database	SQLite (`certaintyai.db`)
Lineage	Metadata Catalog	In-Memory / File
Credentials	Secret Management	Local `.env`
Packaging	Orchestrator	Local scripts
Observability	Telemetry & Logs	Console logs

3. Pre-Requisites

Before executing the deployment, ensure the following environments and tools are configured:

Cloud Environment Pre-requisites:

- ****Google Cloud Platform (GCP) Project**** with billing enabled.
- ****Service Account**** with the following IAM roles:
 - `Secret Manager Secret Accessor`
 - `Cloud SQL Client`
 - `Kubernetes Engine Admin`
 - `Storage Object Creator / Viewer`
- ****VPC Networking:**** Established private subnets with a NAT Gateway configured for secure external LLM API egress if local GCP models are not used.

Local Administration Machine Pre-requisites:

- ****Docker Desktop**** (version 24.0.0+) or Docker Engine.
- ****Google Cloud SDK (`gcloud` CLI)**** installed and authenticated.
- ****Kubernetes CLI (`kubectl`)**** and ****Helm**** (version 3.0+) installed.
- ****Node.js**** (v18.0.0+) and ****Python**** (v3.10+) for local packaging validation.

4. Step-by-Step Containerization & Orchestration Guide

Step 4.1: Create Container Manifests

To support enterprise deployment, we construct multi-stage Dockerfiles for both the frontend and backend.

1. Backend Dockerfile (`backend/Dockerfile`)

Create a high-security, non-root, multi-stage Python container build:

```
# Stage 1: Build dependencies
FROM python:3.10-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev gcc && \
    rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# Stage 2: Operational runtime
FROM python:3.10-slim AS runner
WORKDIR /app

RUN groupadd -g 999 appuser && \
    useradd -r -u 999 -g appuser appuser

COPY --from=builder /root/.local /home/appuser/.local
COPY --chown=appuser:appuser . .

ENV PATH=/home/appuser/.local/bin:$PATH
ENV PYTHONUNBUFFERED=1
USER appuser

EXPOSE 8000
CMD ["gunicorn", "-k", "uvicorn.workers.UvicornWorker", "-c", "gunicorn_conf.py", "app.main:app"]
```

2. Frontend Dockerfile (`frontend/Dockerfile`)

Create a lightweight, highly optimized web server container for static asset delivery:

```
# Stage 1: Build React/Vite application
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Serve static files via production Nginx
FROM nginx:1.25-alpine AS runner
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Step 4.2: Local Production Simulation (Docker Compose)

Create a secure `docker-compose.prod.yml` file to simulate and validate the production architecture locally before cloud deployment:

```
version: '3.8'
```

```
services:
```

```

database:
  image: postgres:15-alpine
  container_name: certaintyai-postgres
  environment:
    POSTGRES_DB: certaintyai
    POSTGRES_USER: certainty_admin
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data
  networks:
    - certainty-net
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U certainty_admin -d certaintyai"]
    interval: 5s
    timeout: 5s
    retries: 5

backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: certaintyai-backend
  environment:
    - DATABASE_URL=postgresql://certainty_admin:${DB_PASSWORD}@database:5432/certaintyai
    - LLM_PROVIDER=anthropic
    - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    - JWT_SECRET=${JWT_SECRET}
  ports:
    - "8000:8000"
  depends_on:
    database:
      condition: service_healthy
  networks:
    - certainty-net

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: certaintyai-frontend
  ports:
    - "80:80"
  depends_on:
    - backend
  networks:
    - certainty-net

networks:
  certainty-net:
    driver: bridge

volumes:
  pgdata:

```

Step 4.3: Deploying to Google Kubernetes Engine (GKE)

Step 1: Authenticate and Configure GCP

Configure your local environment to target your enterprise Google Cloud project:

```
# Log in to Google Cloud
gcloud auth login

# Set your active project ID
gcloud config set project YOUR_PROJECT_ID

# Enable required cloud service APIs
gcloud services enable \
  container.googleapis.com \
  sqladmin.googleapis.com \
  secretmanager.googleapis.com
```

Step 2: Push Images to Artifact Registry

Create a repository and upload your enterprise Docker containers:

```
# Create Artifact Registry Repository
gcloud artifacts repositories create certaintyai-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="CertaintyAI Enterprise Container Images"

# Authenticate Docker to GCP
gcloud auth configure-docker us-central1-docker.pkg.dev

# Tag and Push Backend
docker tag certaintyai-backend us-central1-docker.pkg.dev/YOUR_PROJECT_ID/certaintyai-repo/backend:v1
docker push us-central1-docker.pkg.dev/YOUR_PROJECT_ID/certaintyai-repo/backend:v1

# Tag and Push Frontend
docker tag certaintyai-frontend us-central1-docker.pkg.dev/YOUR_PROJECT_ID/certaintyai-repo/frontend:v1
docker push us-central1-docker.pkg.dev/YOUR_PROJECT_ID/certaintyai-repo/frontend:v1
```

Step 3: Provision GKE Cluster and SQL

Provision the infrastructure utilizing secure, isolated networks:

```
# Create a GKE Autopilot Cluster (production-ready scaling by default)
gcloud container clusters create-auto certaintyai-cluster \
  --region=us-central1 \
  --network=default

# Create highly available Cloud SQL PostgreSQL instance
gcloud sql instances create certaintyai-db-prod \
  --database-version=POSTGRES_15 \
  --tier=db-custom-2-7680 \
  --region=us-central1 \
  --availability-type=REGIONAL
```

Step 4: Configure Enterprise Secrets

Save sensitive API keys and cryptographic secrets directly into Google Secret Manager:

```
# Save LLM API Key
echo -n "your-anthropic-api-key" | gcloud secrets create ANTHROPIC_API_KEY --data-file=-
```

```
# Save DB Password
echo -n "secure-db-password" | gcloud secrets create DB_PASSWORD --data-file=-

# Save JWT Encryption Key
echo -n "cryptographic-jwt-secret-key" | gcloud secrets create JWT_SECRET --data-file=-
```

Step 5: Kubernetes Manifest Orchestration

Deploy the cluster using standard Helm charts or direct Kubernetes YAML manifests (`deployment.yaml`). This mounts secret keys securely using the GCP Secret Manager CSI Driver.

5. Security & Isolation Hardening (GCP Compliance)

To achieve full compliance in healthcare (HIPAA) and financial (BFSI) environments:

1. **Private Service Connect (Database Isolation):**

Configure Cloud SQL to use Private IP addresses only. Disable public IPv4 access entirely, linking your GKE cluster directly via Private Service Connect inside the VPC.

2. **VPC Service Controls:**

Establish a security perimeter around your Vertex AI models, preventing exfiltration of enterprise knowledge graphs or customer assessment records.

3. **Data Encryption (CMEK):**

Utilize Cloud Key Management Service (KMS) with Customer-Managed Encryption Keys to encrypt the PostgreSQL database volumes at rest.

4. **W3C Semantic Provenance Validation:**

Ensure Layer 8's Audit Trail Database cryptographically signs audit logs before writing back to storage, enabling third-party auditors to verify that data lineages were not altered post-facto.

6. Ontology-Driven GraphRAG Ingest & Architecture (Track 3 Ready)

To support high-concurrency enterprise queries and satisfy the **95%+ factual accuracy** required for regulated audits, CertaintyAI implements a hybrid **Ontology-Driven GraphRAG** pipeline in production:

6.1. Dynamic Storage Layer Config

1. **Relational + Vector Indexing:** Powered by **PostgreSQL** with the **pgvector** extension. All document chunk embeddings are indexed using **HNSW (Hierarchical Navigable Small World)** on cosine distance (`vector_cosine_ops`), enabling **<5ms** retrieval times across millions of records.

2. **Semantic Graph Storage:** Powered by **Neo4j Enterprise**, maintaining the rigid domain entities (`Patient`, `Student`, `AML Alert`) and W3C-aligned relationship edges. Connected via custom bolt connection pooling in

the API gateway.

6.2. Document Chunking & Processing Pipelines

- **Semantic Chunking:** Documents are split dynamically at natural semantic transitions by calculating the cosine similarity between consecutive sentences, preventing disjointed snippets.
- **Hierarchical Parent-Child Association:** Documents are indexed in child fragments (128 tokens) for precise vector matching, but retrieved with their parent context (512 tokens) to supply the LLM with complete local context.

6.3. Embeddings & Search Hybridization

- **Dense Vectors:** Embedded via Google Vertex AI `text-multilingual-embedding-002` (768 dimensions) for multi-language semantic matching.
- **Lexical Recovery:** Merged with **SPLADE v2** sparse token index matches via **Reciprocal Rank Fusion (RRF)** to prevent key database identifiers and acronyms from being missed during searches.

6.4. Security Metadata Tagging Schema

Every document chunk is cataloged with a metadata payload containing `classification` and `allowed_roles` (e.g., `["CISO", "Compliance_Officer"]`). At query-time, the retriever automatically appends role-based filters to the SQL query to block horizontal data leakage and ensure strict multi-tenant privacy.